

# RL-Assisted MPC Algorithms

Horizon, Weight, Residual, Dynamic-Matrix, and SG-TD3 Supervisors

**A. H. Hamed**

Supervisor: P. Mhaskar

Department of Chemical Engineering, McMaster University

Polymer CSTR and Aspen Dynamics  $C_2$  splitter

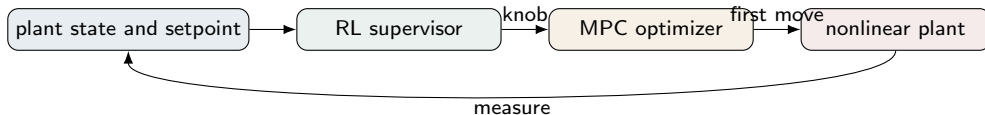
# The design principle is RL around MPC, not RL instead of MPC

## Fixed MPC is structured

- It solves a constrained finite-horizon optimization problem.
- It respects input bounds and move limits.
- It is interpretable through model, horizon, and cost weights.

## Naive RL is flexible but risky

- It can adapt to nonlinear closed-loop behavior.
- It can also propose unsafe or poorly scaled input moves.
- Pure actor actions do not automatically satisfy process constraints.



**Takeaway:** RL learns which MPC design knob to change while MPC remains the constrained move generator.

# Common notation for all supervisors

## Plant and estimator coordinates

$$x_{a,k+1} = A_{\text{aug}}x_{a,k} + B_{\text{aug}}\Delta u_k,$$

$$y_k = C_{\text{aug}}x_{a,k},$$

$$\hat{x}_{a,k+1} = A_{\text{aug}}\hat{x}_{a,k} + B_{\text{aug}}\Delta u_k + L(y_k - \hat{y}_k).$$

- $x_a$  is the augmented offset-free state.
- $u$  is manipulated input and  $\Delta u_k = u_k - u_{k-1}$ .
- $y$  is controlled output.
- Calculations use scaled deviation variables unless marked physical.

## Receding-horizon MPC objective

$$J_k = \sum_{i=1}^{N_p} e_{k+i}^T Q e_{k+i} + \sum_{i=0}^{N_c-1} \Delta u_{k+i}^T R \Delta u_{k+i},$$

$$e_{k+i} = y_{k+i} - y_{k+i}^{\text{sp}},$$

$$\Delta U_k^* = \arg \min_{\Delta U_k} J_k \quad \text{s.t.} \quad u_{\min} \leq u_{k+i} \leq u_{\max}.$$

- $N_p$  is prediction horizon.
- $N_c$  is control horizon.
- Only the first optimized move is applied.

**Takeaway:** Every RL method changes a different term in this same MPC loop.

# The algorithm families are different ways to change MPC

| Family         | RL action                            | MPC quantity changed                      | Typical agent          |
|----------------|--------------------------------------|-------------------------------------------|------------------------|
| Horizon        | discrete recipe index                | $N_p, N_c$                                | DQN or dueling DQN     |
| Weights        | four continuous multipliers          | $Q, R$ penalties                          | TD3, SAC, SG-TD3       |
| Residual       | continuous correction in input space | final input after MPC                     | TD3, SAC, TD7, SG-TD3  |
| Dynamic matrix | continuous correction vector $z$     | Markov blocks $M_i$ and lifted matrix $G$ | TD3, SG-TD3            |
| Combined       | multiple actions                     | horizon, weights, model, residual         | multi-agent supervisor |

## Two process settings

- Polymer CSTR: controlled outputs  $(\eta, T)$ , manipulated inputs  $(Q_c, Q_m)$ .
- Distillation column: tray-24 ethane composition and tray-85 temperature, manipulated inputs reflux flow and reboiler duty.

**Takeaway:** The methods are interpretable because each action coordinate has a controller meaning.

# Learning signal: the agent sees closed-loop reward, not only one-step error

## Generic reward form

$$r_k = -e_k^\top Q_r e_k - \Delta u_k^\top R_r \Delta u_k + b_{\text{near}}(e_k) - p_{\text{viol}}(k),$$
$$G_t = \sum_{\ell=0}^{\infty} \gamma^\ell r_{t+\ell}.$$

## What the reward encourages

- small output tracking error
- reasonable input movement
- staying near target bands

## What it does not guarantee by itself

- no hard safety certificate
- no guaranteed constraint satisfaction for raw actor output
- no guarantee that short-term prediction score matches long-term return

**Takeaway:** Reward defines the learning objective, while MPC and safety gates define the executable controller envelope.

# Horizon supervisor: RL changes how far MPC looks ahead

## Discrete action table

$$\mathcal{A}_H = \{(N_p^{(j)}, N_c^{(j)}) : N_c^{(j)} \leq N_p^{(j)}\},$$
$$a_k = j, \quad (N_p, N_c)_k = \mathcal{A}_H[j].$$

- Polymer grid:  $N_p \in \{8, \dots, 19\}$ ,  $N_c \in \{3, \dots, 9\}$ .
- Distillation grid:  $N_p \in \{4, \dots, 14\}$ ,  $N_c \in \{2, \dots, 13\}$ .
- Invalid choices with  $N_c > N_p$  are removed.

## Why horizon matters

- Larger  $N_p$ : sees slower settling and delayed effects.
- Smaller  $N_p$ : cheaper optimization and more local behavior.
- Larger  $N_c$ : gives MPC more move-shaping freedom.
- Smaller  $N_c$ : makes input plans simpler and usually smoother.
- DQN is natural because the action is a finite recipe index.



**Takeaway:** The horizon agent changes the time scale of MPC, not the plant input directly.

# Horizon mathematics: DQN learns a value for each horizon recipe

## Action-value learning

$$\begin{aligned}Q_{\psi}(s_k, a_k) &\approx \mathbb{E} \left[ \sum_{\ell=0}^{\infty} \gamma^{\ell} r_{k+\ell} \mid s_k, a_k \right], \\y_k^{\text{DQN}} &= r_k + \gamma(1 - d_k) \max_{a'} Q_{\psi}(s_{k+1}, a'), \\ \mathcal{L}_Q &= \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[ \left( Q_{\psi}(s, a) - y^{\text{DQN}} \right)^2 \right].\end{aligned}$$

## Decision rule

$$a_k = \begin{cases} \text{random}(\mathcal{A}_H), & \text{with } \epsilon_k, \\ \arg \max_a Q_{\psi}(s_k, a), & \text{otherwise.} \end{cases}$$

Epsilon decays so early training explores horizons and late training exploits learned recipes.

# Dueling DQN separates state difficulty from action preference

## Dueling horizon value decomposition

$$Q_{\psi}(s, a) = V_{\psi}(s) + A_{\psi}(s, a) - \frac{1}{|\mathcal{A}_H|} \sum_{a' \in \mathcal{A}_H} A_{\psi}(s, a').$$

### State-value stream

- learns whether the current process condition is easy or difficult
- does not need to explain differences between horizon actions

### Advantage stream

- learns which horizon recipe is better in the current state
- helps when many recipes have similar absolute return

**Takeaway:** Dueling DQN is useful when horizon choices are close in value but still different in closed-loop behavior.

# Weight supervisor: RL changes the tradeoff inside MPC

## Nominal and adapted objectives

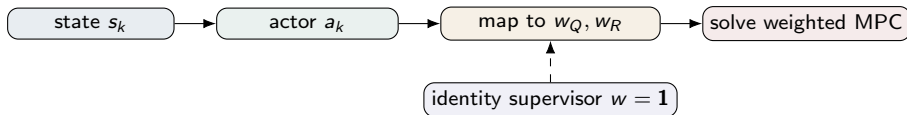
$$\begin{aligned} J_0 &= \sum_{i=1}^{N_p} e_i^\top Q_0 e_i + \sum_{i=0}^{N_c-1} \Delta u_i^\top R_0 \Delta u_i, \\ Q(a_k) &= \text{diag}(w_{Q,k}) Q_0, \quad R(a_k) = \text{diag}(w_{R,k}) R_0, \\ J(a_k) &= \sum_{i=1}^{N_p} e_i^\top Q(a_k) e_i + \sum_{i=0}^{N_c-1} \Delta u_i^\top R(a_k) \Delta u_i. \end{aligned}$$

## Control interpretation

Larger  $w_{Q_i}$  makes MPC track output  $i$  more aggressively. Larger  $w_{R_j}$  makes MPC move input  $j$  more smoothly. Typical multiplier bounds are  $0.75 \leq w_i \leq 2.0$ .

**Takeaway:** Weight adaptation is a tuning policy: the model stays fixed and RL changes controller aggressiveness.

# Weight algorithm: identity is the safe supervisor



## Training transition

$$s_k \rightarrow a_k \rightarrow (Q_k, R_k) \rightarrow \Delta u_k^* \rightarrow y_{k+1},$$
$$\mathcal{D} \leftarrow (s_k, a_k^{\text{exec}}, r_k, s_{k+1}, d_k).$$

Replay stores the executed multiplier action so the critic learns the action that actually produced the next state.

## Safety role

- Identity action recovers nominal MPC weights.
- Nonfinite multipliers or failed solves can fall back to identity.
- SG-TD3 can compare actor weights against identity weights using the same critics.

**Takeaway:** For weights, safety means the controller can always fall back to familiar fixed tuning.

# Residual supervisor: RL adds a bounded correction after MPC

## Residual correction around nominal MPC

$$\begin{aligned}\Delta u_k^{\text{mpc}} &= \text{MPC}(s_k, y_k^{\text{sp}}), \\ \delta u_k^{\text{res}} &= \delta_{\min} + \frac{a_k + 1}{2} \odot (\delta_{\max} - \delta_{\min}), \\ \Delta u_k^{\text{exec}} &= \text{clip}(\Delta u_k^{\text{mpc}} + \rho_k \delta u_k^{\text{res}}, \Delta u_{\min}, \Delta u_{\max}).\end{aligned}$$

### Meaning of each term

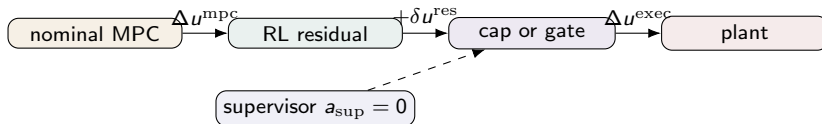
- $\Delta u^{\text{mpc}}$ : stabilizing baseline move.
- $\delta u^{\text{res}}$ : learned local correction.
- $\rho_k$ : optional authority factor.
- Clipping enforces input or move limits.

### Typical correction bounds

- Polymer residual action: roughly  $[-0.25, 0.25]^2$  in scaled move coordinates.
- Distillation residual action: roughly  $[-0.02, 0.02]^2$ .
- The correction is intentionally smaller than direct input authority.

**Takeaway:** Residual RL is local adaptation: MPC supplies the main move and RL repairs what MPC misses.

# Residual safety: zero residual is the natural fallback



## Authority scheduling

$$\rho_k = \rho_{\min} + (1 - \rho_{\min})g_k,$$
$$g_k \in [0, 1].$$

The schedule can release authority gradually after warm start, or use a mismatch confidence signal.

## Guard logic

- zero residual gives nominal MPC
- early-release guard can shrink the residual
- nonfinite output falls back to zero residual
- replay stores the residual that was actually executed

**Takeaway:** The residual family has a built-in safe action: do nothing beyond MPC.

# Dynamic-matrix supervisor: Markov parameters are the prediction model

## Impulse-response or Markov blocks

$$M_i = C_{\text{aug}} A_{\text{aug}}^{i-1} B_{\text{aug}}, \quad i = 1, \dots, N_p,$$
$$M_i \in \mathbb{R}^{n_y \times n_u}.$$

- $M_i$  maps an input move at the current time to output response  $i$  samples later.
- In classical control language, the lifted matrix made from  $M_i$  is the dynamic matrix.
- In the code this is the Markov correction path.



**Takeaway:**  $M_i$  is not a neural-network parameter. It is a physical lifted-response coefficient used by MPC.

# Dynamic matrix $G$ : stacking Markov blocks gives lifted prediction

## Lifted prediction equation

$$Y_k = Y_k^{\text{free}} + G_0 \Delta U_k,$$
$$Y_k = \begin{bmatrix} y_{k+1} \\ y_{k+2} \\ \vdots \\ y_{k+N_p} \end{bmatrix}, \quad \Delta U_k = \begin{bmatrix} \Delta u_k \\ \Delta u_{k+1} \\ \vdots \\ \Delta u_{k+N_c-1} \end{bmatrix}.$$

## Why this form matters

- $Y^{\text{free}}$  is the output response from the current estimated state with no new input moves.
- $G_0 \Delta U$  is the future output contribution caused by planned input moves.
- Changing  $G$  changes how MPC believes each move will affect future outputs.

**Takeaway:** The Markov method adapts the matrix that converts planned input moves into predicted output trajectories.

# The dynamic matrix is a block-Toeplitz matrix

$$G_0 = \begin{bmatrix} M_1 & 0 & 0 & \cdots & 0 \\ M_2 & M_1 & 0 & \cdots & 0 \\ M_3 & M_2 & M_1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ M_{N_p} & M_{N_p-1} & M_{N_p-2} & \cdots & M_{N_p-N_c+1} \end{bmatrix}.$$

## Reading one block column

The first block column tells how the current move  $\Delta u_k$  affects all future outputs. The second block column tells how the next planned move  $\Delta u_{k+1}$  affects outputs one sample later, and so on.

## Why MPC likes this structure

The entire prediction over  $N_p$  samples becomes one linear map from the planned move sequence  $\Delta U_k$  to the stacked output sequence  $Y_k$ . This is why changing  $M_i$  changes the optimizer's future view.

**Takeaway:** The dynamic matrix is the finite-horizon input-output model seen by MPC.

# What is $z$ ? A bounded correction coordinate for the dynamic matrix

## Basis expansion of Markov blocks

$$M_i(z_k) = M_{i,0} + \sum_{j=1}^r z_{j,k} B_i^{(j)},$$

$$G(z_k) = \text{Toeplitz} \left( M_1(z_k), \dots, M_{N_p}(z_k) \right),$$

$$z_k \in [-z_{\max}, z_{\max}]^r.$$

## Meaning of $z_j$

- $z_j = 0$ : nominal dynamic matrix.
- $z_j > 0$ : strengthen the selected basis direction.
- $z_j < 0$ : weaken or reverse that basis direction.
- $z_{\max}$  limits how far the model can move.

**Takeaway:**  $z$  is the RL or LS action in Markov space: it edits the prediction model, then MPC still computes the input.

# Least-squares Markov correction supplies a supervisor action

Online LS fit over a recent prediction window

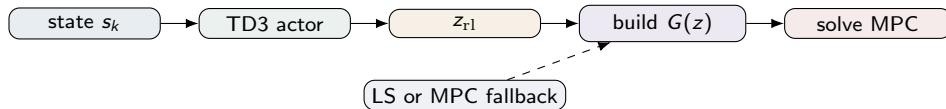
$$z_k^{\text{LS}} = \arg \min_z \sum_{\tau \in \mathcal{W}_k} \left\| Y_\tau^{\text{meas}} - \left( Y_\tau^{\text{free}} + G(z) \Delta U_\tau^{\text{real}} \right) \right\|_{W_y}^2 + \lambda_z \|z\|_2^2,$$
$$\mathcal{W}_k = \{\tau : \tau \text{ has } N_p \text{ future measurements}\}.$$

Prediction-improvement score

$$S_{\text{pred}}(z) = \sum_{\tau \in \mathcal{W}_k} \left( \|W_y E_0(\tau)\|_2^2 - \|W_y E_z(\tau)\|_2^2 \right) - \lambda_z \|z\|_2^2,$$
$$E_0(\tau) = Y_\tau^{\text{meas}} - \left( Y_\tau^{\text{free}} + G_0 \Delta U_\tau^{\text{real}} \right),$$
$$E_z(\tau) = Y_\tau^{\text{meas}} - \left( Y_\tau^{\text{free}} + G(z) \Delta U_\tau^{\text{real}} \right).$$

**Takeaway:** LS is a data-driven local model correction, useful as teacher, fallback, or SG-TD3 supervisor.

## Markov TD3: the actor proposes $z$ , MPC executes with $G(z)$



### Runtime candidate evaluation

$$\begin{aligned} z_{rl} &= z_{\max} \text{clip}(a_{rl}, -1, 1), \\ U_{rl}^* &= \arg \min_U J(U; G(z_{rl})), \\ \text{accept} &\Leftrightarrow S_{\text{pred}}(z_{rl}) > s_{\min} \quad \text{and} \quad d_{\text{gain}}(z_{rl}) \leq d_{\max} \quad \text{and} \quad \Delta J_{\text{nom}} \leq \epsilon. \end{aligned}$$

- Guarded Markov TD3 can execute TD3, fall back to LS, or fall back to nominal MPC.
- The SG-TD3 Markov wrapper uses LS-or-MPC as the supervisor during warm start and critic warm.

**Takeaway:** Markov control learns a model-correction action, then relies on MPC to translate that model into inputs.

# Standard TD3 is the continuous-control base agent

## Actor and twin critics

$$a_k = \mu_\theta(s_k) + \eta_k,$$
$$Q_{\phi_1}(s, a), \quad Q_{\phi_2}(s, a).$$

The actor proposes a normalized action. The critics estimate expected return for state-action pairs.

## Critic and actor losses

$$y_k = r_k + \gamma(1 - d_k) \min_{i=1,2} Q_{\bar{\phi}_i}(s_{k+1}, \tilde{a}_{k+1}),$$
$$\mathcal{L}_{Q_i} = \mathbb{E} [(Q_{\phi_i}(s_k, a_k) - y_k)^2],$$
$$\mathcal{L}_\pi = -\mathbb{E} [Q_{\phi_1}(s_k, \mu_\theta(s_k))].$$

## Target-policy smoothing

$$\tilde{a}_{k+1} = \text{clip}(\mu_{\bar{\theta}}(s_{k+1}) + \epsilon, -a_{\max}, a_{\max}).$$

## TD3 protections

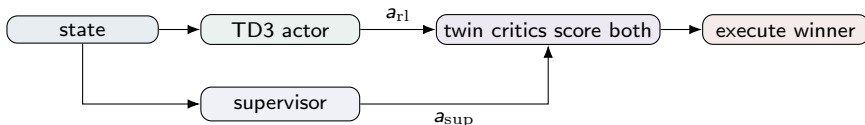
- clipped double-Q target reduces overestimation
- delayed actor update stabilizes critic learning
- target noise smooths sharp critic errors

**Takeaway:** TD3 stabilizes continuous action learning, but it still executes only the actor candidate unless we add a supervisor gate.

# SG-TD3 adds a second candidate action before execution

Two candidates in the same normalized action space

$$\begin{aligned}a_{\text{rl},k} &= \mu_{\theta}(s_k), \\a_{\text{sup},k} &= \mu_{\text{sup}}(s_k), \\a_{\text{rl},k}, a_{\text{sup},k} &\in [-a_{\text{max}}, a_{\text{max}}]^{n_a}.\end{aligned}$$



Execution difference from standard TD3

Standard TD3 executes  $a_{\text{rl}}$  whenever the policy is live. SG-TD3 evaluates  $a_{\text{rl}}$  against  $a_{\text{sup}}$  first, then executes the selected candidate and stores that executed action in replay.

# SG-TD3 scoring: critic value minus uncertainty and motion penalties

Conservative score used by the gate

$$\begin{aligned}Q_{\min}(s, a) &= \min(Q_1(s, a), Q_2(s, a)), \\D_Q(s, a) &= |Q_1(s, a) - Q_2(s, a)|, \\S(s, a) &= Q_{\min}(s, a) - \rho_Q D_Q(s, a) - \kappa_{\text{prev}} \|a - a_{\text{prev}}\|_2^2 - \kappa_{\text{sup}} \|a - a_{\text{sup}}\|_2^2.\end{aligned}$$

Gate rule

$$a_k^{\text{exec}} = \begin{cases} a_{\text{rl},k}, & S(s_k, a_{\text{rl},k}) > S(s_k, a_{\text{sup},k}) + \epsilon_A, \\ a_{\text{sup},k}, & \text{otherwise.} \end{cases}$$

Current critic-warm Markov settings

- $\rho_Q = 0.5$
- $\kappa_{\text{sup}} = 0.02$  for polymer Markov and new distillation Markov wrapper
- $\kappa_{\text{prev}} = 0.01$
- $\epsilon_A = 0$ , default-to-supervisor on ties

**Takeaway:** The gate rewards high value, penalizes critic disagreement, and avoids unnecessary jumps away from safe actions.

# SG-TD3 training: the critic learns executed actions

Replay stores the action that actually moved the plant

$$\mathcal{D} \leftarrow (s_k, a_k^{\text{exec}}, r_k, s_{k+1}, d_k, a_{\text{rl},k}, a_{\text{sup},k}, a_{\text{prev},k}, \text{source}_k).$$

Counterfactual actor and supervisor actions are metadata. The critic sees the action that produced the next state.

Critic target remains standard TD3

$$y_k = r_k + \gamma(1 - d_k) \min_i Q_{\tilde{\phi}_i}(s_{k+1}, \tilde{a}_{k+1}),$$
$$\mathcal{L}_{Q_i} = \mathbb{E} [(Q_{\phi_i}(s_k, a_k^{\text{exec}}) - y_k)^2].$$

**Takeaway:** SG-TD3 improves safety through execution selection first, not by inventing counterfactual critic targets.

# SG-TD3 can also regularize the actor toward the supervisor

## Optional supervisor-aware actor objective

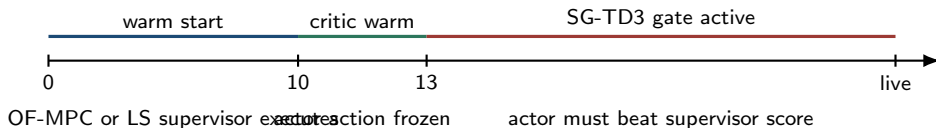
$$\mathcal{L}_\pi = -\mathbb{E}[Q_{\text{mode}}(s, \mu_\theta(s))] + \lambda_{\text{sup}} \mathbb{E} \left[ w_{\text{sup}} \|\mu_\theta(s) - a_{\text{sup}}\|_2^2 \right] + \lambda_{\Delta a} \mathbb{E} \left[ \|\mu_\theta(s) - a_{\text{prev}}\|_2^2 \right],$$
$$w_{\text{sup}}(s) = \sigma \left( \frac{\epsilon_A - [S(s, a_{\text{rl}}) - S(s, a_{\text{sup}})]}{\tau_{\text{sup}}} \right).$$

## Interpretation

- $w_{\text{sup}}$  becomes large when the actor does not beat the supervisor by the margin.
- $\lambda_{\text{sup}}$  controls imitation pressure toward the supervisor.
- Current critic-warm wrappers use  $\lambda_{\text{sup}} = 0$ , so this term is documented but disabled.

**Takeaway:** Actor regularization is optional. The current safety gain is mainly from the execution gate.

# Warm start and critic warm separate data collection from actor release



## Why warm start

- keeps early plant operation under OF-MPC or LS supervisor
- fills replay with meaningful closed-loop transitions
- prevents random actor actions from defining early process behavior

## Why critic warm

- lets critics train before actor execution matters
- keeps the gate from trusting untrained Q-values too early
- mirrors the polymer Markov and new distillation Markov SG-TD3 design

**Takeaway:** The schedule makes SG-TD3 a gradual release mechanism, not an immediate replacement for MPC.

# How SG-TD3 is adapted to each method family

| Family   | Actor action                           | Supervisor action                                          | Safety benefit                                                        |
|----------|----------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------|
| Weights  | $a_{r1} \mapsto [w_Q, w_R]$            | $a_{sup} \mapsto \mathbf{1}$                               | Actor must beat nominal fixed tuning before it changes aggressiveness |
| Residual | $a_{r1} \mapsto \delta u^{\text{res}}$ | $a_{sup} = 0$                                              | Actor must beat nominal-MPC residual before adding correction         |
| Markov   | $a_{r1} \mapsto z_{r1}$                | $a_{sup} \mapsto z_{LS}$ if LS accepted, otherwise $z = 0$ | Actor must beat local model correction or nominal dynamic matrix      |

## Important distinction

SG-TD3 is not a stability proof. It improves operational safety by changing execution, using supervisor-default ties, preserving fallback actions, and logging source fractions.

**Takeaway:** The same gate becomes method-specific because the supervisor action has a different control meaning in each family.

# Safety layers have different jobs

| Layer              | What it protects                         | Typical effect                              |
|--------------------|------------------------------------------|---------------------------------------------|
| MPC constraints    | physical input bounds and feasible moves | hard limits on optimized input              |
| Warm start         | early learning under weak replay         | OF-MPC or LS executes first                 |
| Authority ramp     | sudden post-warm actor release           | gradually increases correction size         |
| Fallback action    | failed solve or nonfinite action         | identity, zero residual, LS, or nominal MPC |
| SG-TD3 gate        | weak critic evidence for actor action    | supervisor wins unless actor proves value   |
| Shadow diagnostics | audit without intervention               | logs what safety layer would have done      |

## Why the latest Markov wrapper turns live manual layers off

The new distillation Markov SG-TD3 critic-warm runner disables live manual safety layers to isolate gate behavior. Shadow diagnostics stay on, so projected or rejected behavior can be audited without changing  $z_{\text{exec}}$ .

**Takeaway:** Safety is not one switch. It is a hierarchy of constraint handling, release scheduling, fallback, gating, and audit logs.

# Putting the algorithms side by side

| Method         | Changed object          | Action dimension              | Default safe action        |
|----------------|-------------------------|-------------------------------|----------------------------|
| Horizon        | $N_p, N_c$              | number of valid recipes       | baseline horizon recipe    |
| Weights        | $Q, R$                  | 4                             | $w = \mathbf{1}$           |
| Residual       | final $\Delta u$        | $n_u$                         | $\delta u = 0$             |
| Dynamic matrix | $G(z)$ through $M_i(z)$ | basis size $r$                | $z = 0$ or $z^{\text{LS}}$ |
| Combined       | several MPC knobs       | mixed discrete and continuous | per-agent fallback         |

## Development path so far

- 1 Start with offset-free MPC as the baseline controller.
- 2 Add one RL supervisor at a time: horizons, weights, residuals, matrix, dynamic matrix.
- 3 Observe where standard TD3 is too aggressive or too suppressed by fallback.
- 4 Add SG-TD3 so the actor competes with a meaningful supervisor action.
- 5 Use critic-warm release and shadow diagnostics to separate learning from intervention.

**Takeaway:** The project is moving from hand-built safety wrappers toward critic-based supervisor selection.

# Why SG-TD3 helps safety, and what remains uncertain

## Safety improvements

- Actor action is not automatically executed.
- Supervisor wins ties and low-confidence cases.
- Critic disagreement is explicitly penalized.
- Warm start and critic warm reduce early random intervention.
- Replay remains transition-correct because it stores executed action.
- Source logs expose how often actor, supervisor, and fallback are used.

## Remaining scientific risks

- Critic scores can still be wrong under distribution shift.
- A conservative gate can under-release the actor.
- A permissive gate can execute poorly valued actions.
- Supervisor quality matters.
- Improved reward is not the same as guaranteed stability.
- Distillation remains more sensitive than polymer because of strong coupling and slow dynamics.

**Takeaway:** SG-TD3 is a safer execution architecture, not a substitute for closed-loop validation.

# Key message

## One controller, many adaptive knobs

The unifying controller is offset-free MPC. The learning problem is not to replace that controller, but to learn which MPC quantity should be adapted online: horizon, cost weights, final residual correction, or the dynamic matrix used for prediction.

## Why dynamic-matrix Markov correction is important

The Markov method edits  $M_i = C_{\text{aug}} A_{\text{aug}}^{i-1} B_{\text{aug}}$ , the finite-horizon response blocks that form  $G$ . This gives the RL agent a model-adaptation action  $z$ , while MPC still performs constrained input optimization.

## Why SG-TD3 is the current safety direction

SG-TD3 changes the TD3 execution rule from actor-only to actor-versus-supervisor. In weights, the supervisor is identity tuning. In residuals, it is zero residual. In Markov, it is LS-or-MPC  $z$ . This directly connects safety to a meaningful control fallback.

**Takeaway:** The research direction is interpretable adaptive MPC with learned release, not black-box direct control.

# Backup Slides

Detailed equations and implementation notes

## Backup: lifted MPC objective in compact form

### Stacked variables

$$\begin{aligned}Y &= Y^{\text{free}} + G\Delta U, \\E &= Y - Y^{\text{sp}}, \\ \Delta U &= [\Delta u_k^\top, \dots, \Delta u_{k+N_c-1}^\top]^\top.\end{aligned}$$

### Quadratic program form

$$\begin{aligned}J(\Delta U) &= E^\top \bar{Q} E + \Delta U^\top \bar{R} \Delta U \\ &= (Y^{\text{free}} + G\Delta U - Y^{\text{sp}})^\top \bar{Q} (Y^{\text{free}} + G\Delta U - Y^{\text{sp}}) + \Delta U^\top \bar{R} \Delta U.\end{aligned}$$

### Weight and dynamic-matrix methods inside the same equation

- Weight RL changes  $\bar{Q}$  and  $\bar{R}$ .
- Markov RL changes  $G$ .
- Horizon RL changes the dimensions of  $Y$ ,  $\Delta U$ ,  $\bar{Q}$ ,  $\bar{R}$ , and  $G$ .

## Backup: raw action mapping

| Family   | Raw action mapping                                                               | Interpretation                    |
|----------|----------------------------------------------------------------------------------|-----------------------------------|
| Weights  | $w = w_{\min} + \frac{a+1}{2} \odot (w_{\max} - w_{\min})$                       | positive cost multipliers         |
| Residual | $\delta u = \delta_{\min} + \frac{a+1}{2} \odot (\delta_{\max} - \delta_{\min})$ | bounded input correction          |
| Markov   | $z = z_{\max} \text{clip}(a, -1, 1)$                                             | bounded dynamic-matrix correction |
| Horizon  | $(N_p, N_c) = \mathcal{A}_H[a]$                                                  | discrete recipe lookup            |

### Why normalize actions

- Neural networks output actions in comparable normalized ranges.
- Controller-specific scaling happens after the actor.
- Bounds are explicit and easy to audit.

## Backup: Markov basis examples

### IO-pair gain

$$B_i^{(p,q)} = E_{p,q} \odot M_{i,0},$$

$z_{p,q}$  : gain of output  $p$   
from input  $q$ .

### Input-channel gain

$$B_i^{(q)} = M_{i,0} D_q,$$

$z_q$  : gain of all  
outputs from input  $q$ .

### Delay-shift

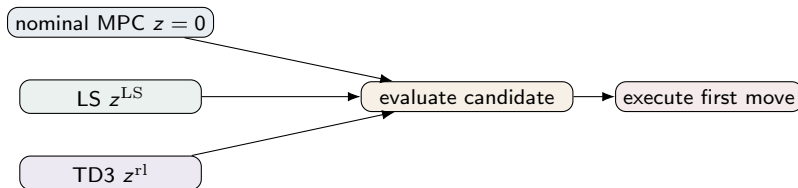
$$B_i \approx M_{i+1,0} - M_{i,0},$$

$z$  : apparent response  
timing correction.

### Why basis design is safety-relevant

The basis determines what model changes are possible. A small  $z$  bound is not enough if the basis direction itself creates harmful coupled gain changes.

## Backup: Markov candidate hierarchy



### Guarded execution

TD3 accepted  $\Rightarrow \Delta U^*(G(z^{\text{r1}}))$ ,  
else LS accepted  $\Rightarrow \Delta U^*(G(z^{\text{LS}}))$ ,  
else  $\Rightarrow \Delta U^*(G_0)$ .

### SG-TD3 execution

The gate compares  $z^{\text{r1}}$  and a supervisor  $z^{\text{sup}}$ . For LS-or-MPC mode,  $z^{\text{sup}} = z^{\text{LS}}$  when LS is accepted and  $z^{\text{sup}} = 0$  otherwise.

# Backup: SG-TD3 source logging

## Why source logs matter

For every closed-loop step, the implementation can log whether the executed action came from warm start, actor, supervisor, fallback, LS, or nominal MPC. These fractions are essential for interpreting learning results.

| Log family               | Question answered                                        |
|--------------------------|----------------------------------------------------------|
| 'sg_selected_source_log' | Did the SG gate choose actor or supervisor?              |
| 'rl_action_source_log'   | Which controller generated the final action?             |
| 'z_executed_log'         | What Markov correction was actually used?                |
| 'shadow_*' diagnostics   | What would disabled safety layers have done?             |
| reward and tracking logs | Did the executed behavior improve the episode objective? |

**Takeaway:** A method can look like RL in code but behave like fallback in the plant. Source logs reveal the true authority.

# Backup: inspected implementation sources

## Reports

- 'report/01\_horizon\_adaptation.md'
- 'report/02\_weight\_adaptation.md'
- 'report/04\_residual\_methods.md'
- 'report/06\_supervisor\_gated\_td3.md'
- 'report/rl\_assisted\_mpc\_design\_summary.md'
- Markov analysis reports from May 2026

## Code

- 'utils/markov\_runner.py'
- 'utils/supervisor\_gated\_action.py'
- 'TD3Agent/supervisor\_gated\_agent.py'
- 'systems/polymer/notebook\_params.py'
- 'systems/distillation/notebook\_params.py'
- active unified runner '.py' files

**Takeaway:** The deck is a method reconstruction from the active runner layer and reports, not a new performance claim.